PROJECT REPORT

RULE-BASED CHATBOT USING PYTHON

Submitted as: Task 1 – AI Internship Program (to create a chatbot)

Submitted by: Prashasti Timsina

College: Birat Multiple College

Technology Used: Python

Development Environment: PyCharm

Submission Date: 2026-10-11

Abstract

This project deals with the creation of a simple rule-based chatbot in Python. A chatbot is a computer program that communicates with an end-user, interpreting their queries and generating appropriate responses. The chatbot created in the described project uses rule-based logic to respond to the user's messages.

The chatbot allows to greet the user and introduce itself, answer simple questions concerning Python, Artificial Intelligence (AI), Machine Learning (ML), and programming, respond to expressions of gratitude and requests for assistance, provide the current date and time, reply to other queries, and end the conversation.

The purpose of the project is to comprehend the underlying principles of chatbots and gain experience in Python programming. The project highlights the basic concepts of chatbots, provides an insight into the principles of Artificial Intelligence, and allows to practice Python basics, including condition statements, loops, string manipulation, and user input.

1. INTRODUCTION

Artificial Intelligence (AI) is a term encompassing a wide range of methodologies related to the development of computer programs which perform tasks requiring human intelligence. One of the manifestations of AI is a chatbot, an application allowing to communicate with a computer using natural language.

A rule-based chatbot is a simplified model of a chatbot which replies to the user's messages basing on a set of previously defined rules. The responses generated by such chatbots depend on the conditions specified by their creators. Contrary to more sophisticated chatbots, rule-based ones cannot evolve on their own, basing their operation on hard-coded instructions. This project aims to create a rule-based chatbot in Python which runs in a terminal and implements a set of basic functions.

2. OBJECTIVES OF THE PROJECT

As mentioned in the introduction, the objectives of the project are to create a simple chatbot in Python and gain an understanding of how such applications work. More specifically, the objectives can be expanded as follows:

- Design and implement a rule-based chatbot in Python.
- Explore the basics of rule-based conversational systems.
- Apply conditional statements to implement responses to the user's messages.
- Gain experience in processing the user's input and practicing string manipulation.
- Provide responses to commonly asked questions concerning Python, Artificial Intelligence (AI), Machine Learning (ML), and programming in general.
- Implement responses to the user's greetings, expressions of gratitude, and requests for assistance.
- Allow the user to request the current date and time.
- Enable the user to exit the chatbot.
- Gain experience in Python programming and testing the created application.

3. PROJECT SCOPE

The project aims to develop a text-based rule-based chatbot which operates in a terminal. The created application is supposed to implement a set of basic functions, allowing to answer the user's questions, respond to the user's greetings and other messages, and exit the conversation. However, the scope of the project does not include the creation of an advanced chatbot which would be able to learn and evolve basing on its conversations with the user.

The created chatbot can be used as a demonstrative example of a rule-based conversational application. The scope of the project includes developing a chatbot which implements the following functions:

- Responding to the user's greetings and introducing the chatbot.
- Answering the user's questions, including those concerned with Python, AI, ML, and programming.
- Responding to the user's requests for assistance.
- Expressing gratitude in case the user has thanked the chatbot.
- Providing the current date and time in case the user has asked for that.
- Allowing to exit the chatbot.

The created rule-based chatbot is a simple one, capable of answering predefined questions and following simple instructions. The chatbot is not able to answer questions which have not been anticipated by the developer.

4. TOOLS AND TECHNOLOGIES USED

4.1 Python

Python is a high-level programming language allowing to create applications using relatively simple code. In the described project, Python is used to implement the chatbot's functionality.

4.2 PyCharm

PyCharm is an Integrated Development Environment (IDE) used to write computer programs. In the described project, PyCharm is used to write Python code and execute it.

4.3 Python Standard Library

The standard Python library contains functions and other constructs enabling the programmer to implement the desired functionality. In the described project, these built-in functions can be used to, for instance, accept the user's input or display it on the screen.

5. SYSTEM REQUIREMENTS

5.1 Hardware Requirements

The following hardware components are required to run the described chatbot:

- A computer or a laptop.
- A keyboard and a monitor.
- A memory card and a hard drive/disk.

5.2 Software Requirements

To run the application described in the project, one needs:

- An operating system (Windows, Linux, macOS).
- Python interpreter.
- PyCharm IDE or another Python code editor.
- A terminal or command-line interface.

6. SYSTEM DESIGN AND WORKING PRINCIPLE

The application described in this project is relatively simple, comprising a few lines of code. The working principle of the created chatbot can be described with the following sequence of operations:

Input

The user enters the message in the terminal window. The message can be of various types: it can be a greeting, a question, a request for assistance, or a command to exit the chatbot.

Processing

The chatbot processes the received message, basing on the code written by the programmer. To make the processing easier, it is possible to convert the input string to lowercase to ensure that the case of the letters does not matter.

Response

In response to the received message, the chatbot outputs the corresponding reply. If the received message matches any of the predefined commands, the reply will be the one corresponding to this command. In other cases, the chatbot will display a default reply, informing the user that it does not understand the input message.

Exit

The chatbot will continue to wait for the user's input until the user sends a message matching the exit command. In response to it, the chatbot will terminate the conversation.

Working Flow

- Start.
- Display a welcome message.
- Get the input from the user.
- Process the input.
- Compare the input to the list of commands.
- If it matches any of them, display the corresponding message.
- If it does not match any of the commands, display the default message.
- End.

7. FEATURES OF THE CHATBOT

7.1 Greeting and Introduction

The chatbot is able to provide a greeting message and introduce itself when the program starts. Moreover, the chatbot can respond to other greeting messages, such as “Hello” or “Hi”.

7.2 Basic Programming Information

The chatbot is able to reply to the user’s questions, providing basic information on Python, AI, ML, and programming in general.

7.3 Artificial Intelligence and Machine Learning

The chatbot is able to give simple explanations on what AI and ML are when the user asks a corresponding question.

7.4 Date and Time

The chatbot can provide the current date and time when the corresponding command is entered.

7.5 Response to Thanks

The chatbot can respond to the user’s expressions of gratitude, such as “Thank you”.

7.6 Help

The chatbot can reply to the user’s request for assistance, providing a list of available commands.

7.7 Unknown Input Handling

In case of receiving an input message which the chatbot does not understand, it will display an appropriate message informing the user of this fact.

7.8 Exit

The user can exit the chatbot by entering an appropriate command.

8. IMPLEMENTATION DETAILS

The described chatbot can be implemented using relatively simple means of Python programming. Below are the details on implementing the chatbot.

8.1 User Input

To get the user's input, the Python built-in input() function can be used.

8.2 String Processing

To make the input string comparable to the reference strings, the string can be converted to lowercase using the lower() method.

8.3 Conditional Statements

The input string can be compared to the reference strings using conditional statements.

8.4 Loops

To make the chatbot run continuously until the exit command is entered, a loop can be used. In Python, loops are implemented using the while keyword.

8.5 Exit

To exit the loop, a break statement can be used.

8.6 Date and Time Handling

To display the current date and time, the datetime module can be used.

The general idea of implementing the chatbot is comparatively simple: any input from the user is matched against a list of reference strings. In case any of the reference strings is matched, the corresponding reply is displayed. In case none of the reference strings is matched, a default reply is displayed.

9. TESTING AND EXPECTED RESULTS

Testing is an essential part of any software development process. The following test cases can be used to check whether the created chatbot works appropriately.

Test Case	Input	Expected Result
1	hello	The chatbot will reply with a greeting message.

Test Case	Input	Expected Result
2	hi	The chatbot will reply with a greeting message.
3	What is Python?	The chatbot will reply with the message containing information on Python.
4	What is AI?	The chatbot will reply with the message containing information on Artificial Intelligence.
5	What is Machine Learning?	The chatbot will reply with the message containing information on Machine Learning.
6	thank you	The chatbot will reply with a message expressing that it is glad to help.
7	An recognised help request	The chatbot will reply with a help message.
8	A message concerning date and time	The chatbot will reply with the current date and time.
9	An unknown question	The chatbot will reply with the corresponding message stating that it does not understand the question.
10	bye	The chatbot will exit.

The expected results given above can be regarded as ideal ones. In reality, some of the cases may require additional coding to produce the expected results. However, it is essential to mark the test cases as failed if the actual results do not match the expected ones.

10. ADVANTAGES OF THE PROJECT

The described project has several advantages, including the following:

- Simplicity: the chatbot can be created using simple Python programming means.
- Easy comprehension: the code of the chatbot is relatively easy to understand.
- Predictability: the chatbot will reply to the user's messages in a predictable manner.
- Learning opportunities: the project can be used to gain experience in Python programming and condition statements.
- Simplicity of use: the chatbot can be operated using a command-line interface.
- Expandability: the chatbot can be easily expanded to provide more functionality.

11. LIMITATIONS OF THE PROJECT

The described project has several limitations, including the following:

- Limited scope: the chatbot will only reply to the user's messages basing on the predefined rules.
- Inflexibility: the chatbot will not understand misspelled words or different phrasings of the same question. This shortcoming is due to the simplicity of the chatbot's design.
- Limited scope: the chatbot will not be able to learn new words or phrases in order to understand the user's messages better.
- Limited depth of understanding: the chatbot will not fully grasp the meaning of the user's messages.
- Limited vocabulary: the chatbot's responses will be limited to the ones provided by the programmer.
- Predictability: the chatbot will always reply with a predefined response to the user's input.
-

12. FUTURE ENHANCEMENTS

The described project can be enhanced in several ways. First of all, more predefined responses can be added. The matching of the input messages to the predefined responses can be made more flexible to account for different phrasings and possible misspellings. Secondly, the chatbot can be made more sophisticated by employing advanced mechanisms of Natural Language Processing (NLP). Such enhancements will make the chatbot more flexible and able to reply to a broader range of messages. Thirdly, the chatbot can be provided with a Graphical User Interface (GUI). Finally, the chatbot can be made accessible via a web interface.

13. CONCLUSION

Working on the Rule-Based Chatbot in Python was my first experience with creating a dialogue system, and I learned a lot of new things. I managed to learn how a simple chatbot could be constructed to understand the user and reply depending on the input.

This project also helped me to practice my Python knowledge and improve my skills. The code of the chatbot could perform several tasks, such as welcoming the user, providing information about Python and artificial intelligence, responding to gratitude and help requests, providing the current date and time, and ending the conversation. Although the chatbot is not perfect and only answers some questions, I am satisfied with the results. This project increased my motivation to improve my Python knowledge and study more complex topics related to Artificial Intelligence and Natural Language Processing.